

Nritya Shakti Academy

Master Developer Architecture Guide



1. Tech Stack & Ecosystem

Frontend (What the user sees):

- React.js (Vite): This is the core library used to build the user interface. It makes the website fast by only updating what needs to change on the screen. We use Vite because it starts the development server instantly.
- TailwindCSS: Instead of writing traditional CSS files, this allows the developer to style buttons and layouts directly by typing class names like "text-red-500".
- React Router: This enables users to visit different pages (like /login or /classes) smoothly without reloading the entire website.
- Framer Motion: Used to create all the beautiful, smooth scroll animations (like the sliding images) which makes the website feel alive.

Backend (How data is managed securely):

- Express.js (Node.js): This acts as the brain of the website in the background, receiving requests from the frontend (like "save this password") and processing them safely.
- MongoDB & Mongoose: This is our database where all student details, passwords, and progress are stored safely like a spreadsheet.
- JWT & Google OAuth: JWT represents digital keys we give to users so they stay logged in securely. Google OAuth lets them sign in via Google directly.
- PDFKit & Nodemailer: PDFKit is uniquely used by instructors to automatically generate downloadable PDF certificates for students! Nodemailer sends OTP emails to users who forget their passwords.

2. Component Matrix & Routing

Routing means deciding what page a user gets to see when they visit a specific URL, and making sure unauthorized people can't access private data.

Public Routes (Open to everyone):

Everything starting from the Home page (/), the About Us page, and the login page. Any guest on the internet can see and interact with these sections.

Protected Dashboards (Hidden behind passwords):

Because our website uses a smart role system in AuthContext.tsx, when a user logs in, they are sent to a completely different layout based on who they are:

- Student Dashboard (/student): For dancers to see their class schedule, mark attendance, watch videos, and get PDF certificates.
- Instructor Dashboard (/instructor): Gives teachers the power to check class attendance and mark courses as 100% complete so students can get certificates.
- Admin Dashboard (/admin): Gives the management full control to manage instructors, manually upload classes, and watch over the entire academy operations instantly.

3. Developer Directory Locator Map

Use this raw map to locate exact component architectures, specifically the Admin Dashboard components and backend routing:

Nritya-Shakti-Academy/

```
%
% % % backend/                                <-- Backend Root
%   % % % controllers/                       <-- API Logic
%   %   % % % adminController.js             <-- Admin Data Endpoints
%   %   % % % authController.js             <-- Login & Password APIs
%   %   % % % certificateController.js
%   % % % models/                           <-- Mongoose DB Entities
%   %   % % % User.js, Class.js, Attendance.js...
%   % % % routes/                           <-- Express Routers
%   %   % % % adminRoutes.js                <-- Admin API Boundaries
%   %   % % % server.js                     <-- Core Express Server Init
%
% % % frontend/                              <-- Frontend Root (Vite/React)
%   % % % src/
%   %   % % % components/
%   %   %   % % % admin/                    <-- Admin Specific Sub-Components
%   %   %   % % % AIChatBot.tsx             <-- Embedded AI Chat System
%   %   %   % % % Navbar.tsx                <-- Universal App Header
%   %   %   % % % ProtectedRoute.tsx        <-- Security Validation Layer
%   %   % % % context/
%   %   %   % % % AuthContext.tsx           <-- Core Stateful Logic
%   %   % % % pages/
%   %   %   % % % AdminDashboard.tsx        <-- Main Admin Control Node
%   %   %   % % % AuthPage.tsx              <-- Authentication/Forgot Pwd
%   %   %   % % % HomePage.tsx              <-- Core Landing Entry
%   %   %   % % % StudentDashboard.tsx
%   %   % % % App.tsx                       <-- Router Combiner
%   % % % public/                           <-- Static Image Assets
```

4. Core Database Models

The database is organized into 6 simple tables (collections) that handle all the data:

1. User Model (User.js):

Stores all account details like Name, Email, and Password. It also checks if a user is an Admin, Instructor, or a Student. For students, it tracks how many classes they finished.

2. Class Model (Class.js):

Think of this as the timetable. It stores information about every dance class happening, including the class name, description, and schedule.

3. Attendance Model (Attendance.js):

The register log. Every time a student attends a class, a record is created here linking the Student with the specific Class and Date.

4. Certificate Model (Certificate.js):

Stores the direct link to the auto-generated PDF certificates that students earn when they complete a major program.

5. Video Model (Video.js):

Holds the links and details (like titles) for all the Masterclass video tutorials that Admins upload for students to watch.

6. Enrollment Model (Enrollment.js):

A simple linking table that keeps track of which student is currently registered into which specific Dance Program.

4. Visual UI Integrations

Below are the core graphical assets implemented within the UI design system mapping:

Ambient Design Layer (Ganesha):



Core Profile Integration Matrix:

